Rejoignez la séance

Scannez le QR code ou entrez le code sur votre téléphone.

COURS

Tester une application

Du test unitaire au test end-to-end

FraisDirect

Une épicerie en ligne qui livre des produits frais



On va la faire grandir dans les prochains cours. Aujourd'hui : s'assurer qu'elle marche.

Une règle métier

La livraison coûte **4,90 €**

Elle est **offerte dès 50 €** d'achat

Un lundi matin

1. Un développeur modifie le calcul des promotions
2. Il teste sa promotion à la main : elle marche
3. Il met en production
4. La livraison redevient payante pour tout le monde
5. On l'apprend une semaine plus tard, par les avis clients

Pourquoi tester ?



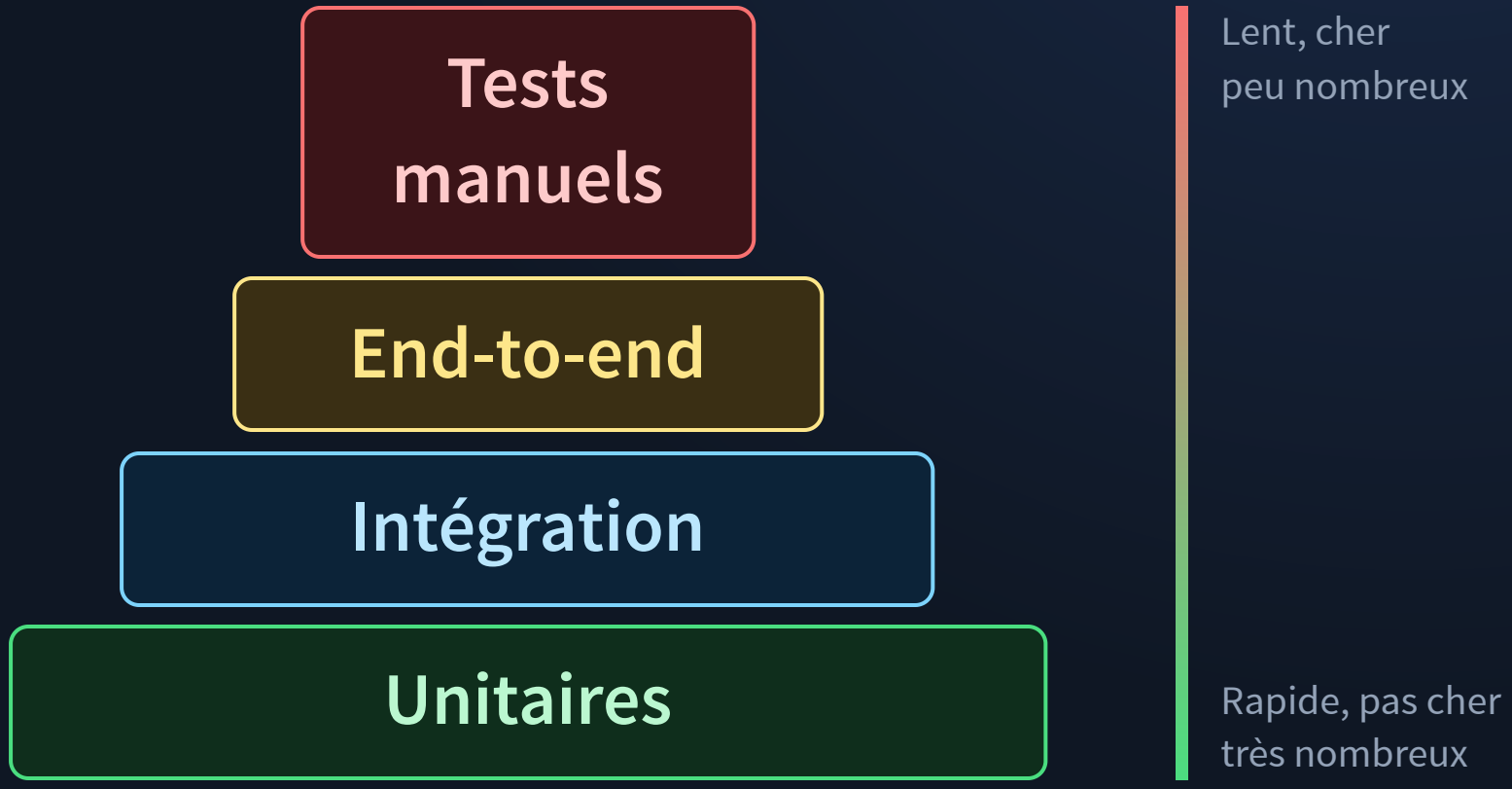
Pourquoi tester ?

- Vérifier que les nouvelles fonctionnalités fonctionnent
- Vérifier qu'il n'y a pas de **régression** : ce qui marchait marche toujours

Au programme

1. Quand écrire ses tests : le **TDD**
2. Tester une classe seule : **tests unitaires**, mocks et fakes
3. Tester avec le framework et la base : **tests d'intégration**
4. Tester comme un utilisateur : **tests end-to-end**

Comment tester ?



Pourquoi on gagne du temps

À la main

1. Démarrer l'application
2. Préparer les données
3. Remplir un panier à 49,99 €
4. Lire les frais de livraison
5. Recommencer à 50 €, à 60 €…

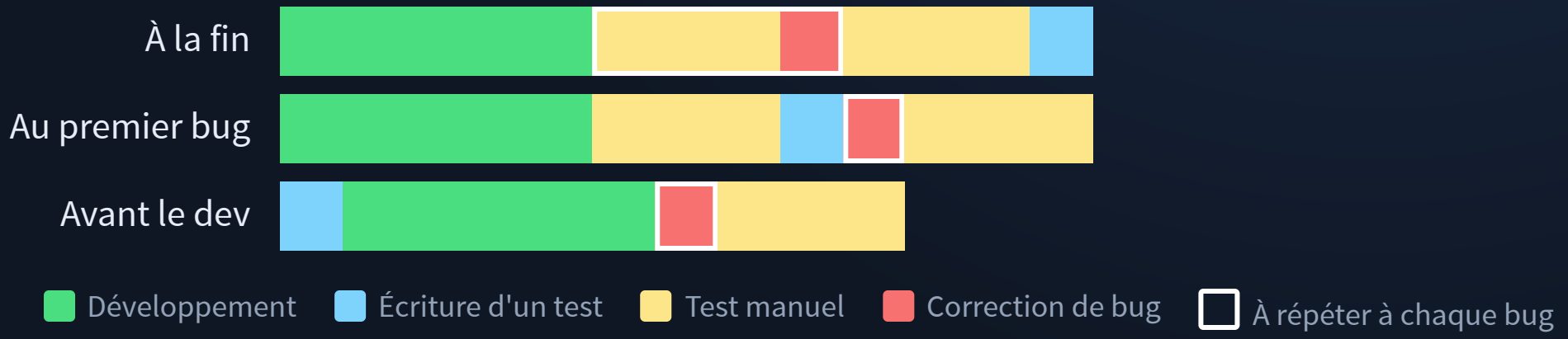
Avec des tests

1. Lancer les tests

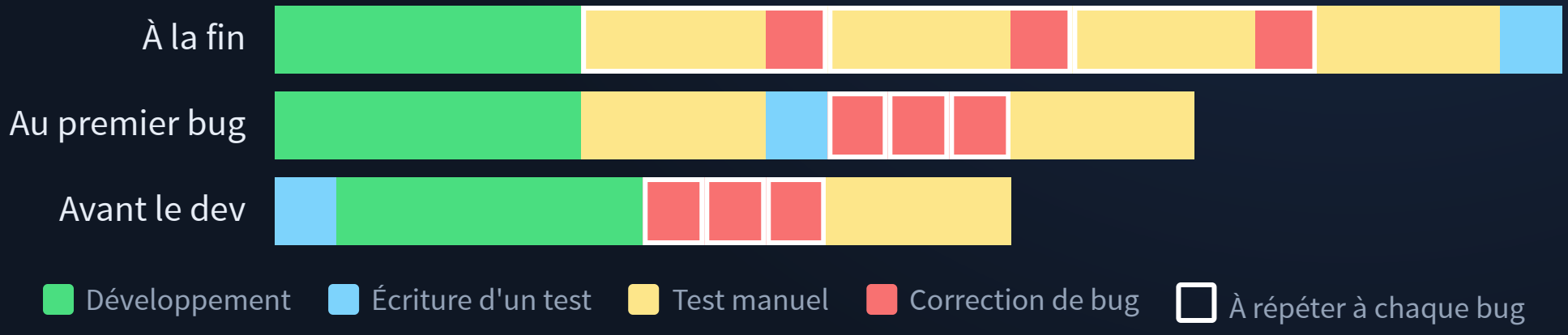
Quelques secondes, tous les cas d'un coup, à chaque modification.

Quand écrire ses tests ?

Une fonctionnalité, un bug



Et si le bug revient trois fois ?



ACTE 1

TDD

Test Driven Development : le test d'abord

QUESTION

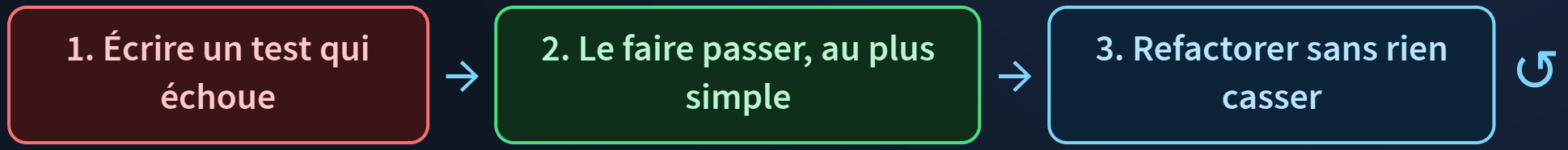
En TDD, par quoi commence-t-on ?

A. Écrire le code de la fonctionnalité

B. Écrire un test qui échoue

C. Refactorer le code existant

Le cycle



- **Rouge** : le test échoue, il décrit ce qu'on veut
- **Vert** : le minimum de code pour qu'il passe
- **Refactor** : on nettoie, les tests protègent

Tour 1 · Rouge

Un panier de 30 € paie la livraison

```java
class FraisLivraisonTest {

    @Test
    void unPanierDe30EurosPaieLaLivraison() {
        // Given : les montants sont en centimes
        long totalPanier = 3000;
        // When
        long frais = new FraisLivraison().calculer(totalPanier);
        // Then
        assertEquals(490, frais);
    }
}
```

Tour 1 · Rouge

On crée la classe, juste pour que ça compile

```
class FraisLivraison {
    long calculer(long totalPanier) {
        return 0;
    }
}
```

```
✗ unPanierDe30EurosPaieLaLivraison
expected: <490> but was: <0>
```

Tour 1 · Vert

Le minimum pour passer au vert

```
1 class FraisLivraison {
2     long calculer(long totalPanier) {
3         return 490;
4     }
5 }
```

✓ unPanierDe30EurosPaieLaLivraison

Tour 2 · Rouge

Un panier de 60 € a la livraison offerte

```
@Test
void unPanierDe60EurosALaLivraisonOfferte() {
    long frais = new FraisLivraison().calculer(6000);

    assertEquals(0, frais);
}
```

```
✗ unPanierDe60EurosALaLivraisonOfferte
expected: <0> but was: <490>
```

Tour 2 · Vert

```
1 class FraisLivraison {
2     long calculer(long totalPanier) {
3         if (totalPanier > 5000) {
4             return 0;
5         }
6         return 490;
7     }
8 }
```

✓ unPanierDe30EurosPaieLaLivraison
✓ unPanierDe60EurosALaLivraisonOfferte

QUESTION

Avec ce code, un panier de pile 50 € a-t-il la livraison offerte ?

```
if (totalPanier > 5000) return 0;
```

Tour 3 · Rouge

On teste la limite

```
@Test
void unPanierDePile50EurosALaLivraisonOfferte() {
    long frais = new FraisLivraison().calculer(5000);

    assertEquals(0, frais);
}
```

```
✗ unPanierDePile50EurosALaLivraisonOfferte
expected: <0> but was: <490>
```

Tour 3 · Vert

```
class FraisLivraison {
    long calculer(long totalPanier) {
        if (totalPanier >= 5000) {
            return 0;
        }
        return 490;
    }
}
```

✓ unPanierDe30EurosPaieLaLivraison
✓ unPanierDe60EurosALaLivraisonOfferte
✓ unPanierDePile50EurosALaLivraisonOfferte

Refactor

On donne un nom aux nombres magiques

```
1  class FraisLivraison {
2      static final long SEUIL_LIVRAISON_OFFERTE = 5000;
3      static final long FRAIS_LIVRAISON = 490;
4
5      long calculer(long totalPanier) {
6          if (totalPanier >= SEUIL_LIVRAISON_OFFERTE) {
7              return 0;
8          }
9          return FRAIS_LIVRAISON;
10     }
11 }
```

✓ 3 tests passent : le refactor n'a rien cassé

Ce qu'on a gagné

- La règle est écrite trois fois, dont deux en langage clair : les noms des tests
- Le cas limite a été trouvé avant la production
- Le prochain qui touche à ce code saura tout de suite s'il casse la règle

Mon (vrai) avis sur le TDD

- ⊕ Essentiel pour le débogage
- ⊕ Pratique pour des POC techniques
- ⊖ Compliqué pour réfléchir à une architecture

ACTE 2

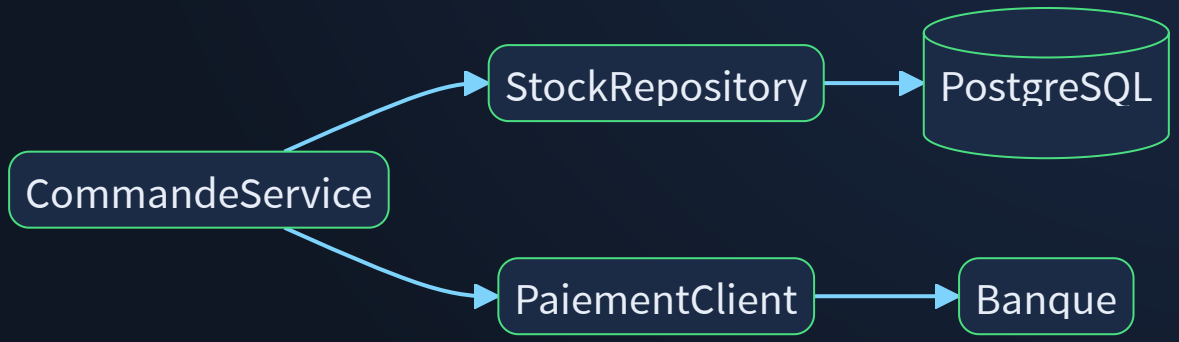
Les tests unitaires

Tester une classe, et seulement elle

Passer une commande

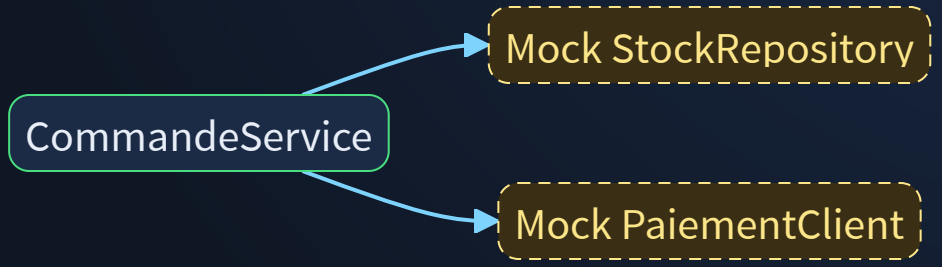
```
class CommandeService {
    void passerCommande(Commande commande) {
        String produit = commande.produit();
        int disponible = stockRepository.quantiteDisponible(produit);
        if (disponible < commande.quantite()) {
            throw new StockInsuffisantException(produit);
        }
        paiementClient.payer(commande.client(),
                commande.montant());
    }
}
```

Le problème



- Il faut une base remplie avec le bon stock
- Il faut une banque… qui débite vraiment
- Et on veut juste tester un `if`

La solution : des faux



- Un **mock** est un faux objet
- On lui dit quoi répondre
- On vérifie ensuite comment il a été appelé

C'est quoi un mock ?



Étape 1 · Créer les faux

```java
@ExtendWith(MockitoExtension.class)
class CommandeServiceTest {

    @Mock
    private StockRepository stockRepository;
    @Mock
    private PaiementClient paiementClient;

    @InjectMocks
    private CommandeService commandeService;
}
```

- `@Mock` crée un faux qui ne fait rien
- `@InjectMocks` crée le vrai service et lui donne les faux

Étape 2 · Programmer le faux

```
// Given : il ne reste que 2 tomates
when(stockRepository.quantiteDisponible("tomate"))
        .thenReturn(2);
Commande commande = new Commande("alice", "tomate", 3, 1200);
```

« Quand on te demande le stock de tomates, réponds 2. »

Étape 3 · Vérifier le résultat

```
// When / Then : la commande est refusée
assertThrows(StockInsuffisantException.class,
        () -> commandeService.passerCommande(commande));
```

Le service a bien appelé le faux, reçu 2, et refusé.

Étape 4 · Vérifier les appels

```
// Then : Alice n'a pas été débitée
verify(paiementClient, never()).payer(any(), anyLong());
```

Le résultat ne suffit pas : on vérifie aussi ce qui **ne** s'est **pas** passé.

Le test complet

```java
@Test
void refuseLaCommandeSiLeStockManque() {
    when(stockRepository.quantiteDisponible("tomate"))
            .thenReturn(2);
    Commande commande = new Commande("alice", "tomate", 3, 1200);

    assertThrows(StockInsuffisantException.class,
            () -> commandeService.passerCommande(commande));

    verify(paiementClient, never()).payer(any(), anyLong());
}
```

Le piège : tester le mock

```
when(additionService.add(2, 3)).thenReturn(5);

assertEquals(5, calculator.calculate(2, 3));
```

- ➖ `calculate` ne fait que renvoyer ce que dit le service
- ➖ On vérifie que le mock renvoie… ce qu'on lui a dit de renvoyer
- ➕ Un test unitaire vérifie une **décision** de la classe : un `if`, un calcul, un appel ou un non-appel

QUESTION

Pour vérifier qu'Alice n'a pas été débitée, de quoi a-t-on besoin ?

A. D'un compte de test à la banque

B. D'un mock de `PaiementClient` et de `verify`

C. D'un `assertEquals` sur le retour de `passerCommande`

C'est quoi un fake ?

- Une vraie implémentation, mais simplifiée
- Exemple : un repository qui stocke dans une `Map`

```
class StockRepositoryEnMemoire implements StockRepository {
    private final Map<String, Integer> stock = new HashMap<>();

    void ajouter(String produit, int quantite) {
        stock.put(produit, quantite);
    }

    @Override
    public int quantiteDisponible(String produit) {
        return stock.getOrDefault(produit, 0);
    }
}
```

Mock ou fake ?

	Mock	Fake
Comment	Mockito, dans chaque test	Une classe écrite une fois
Répond	Ce qu'on lui a programmé	Comme le vrai, en simplifié
Vérifie les appels	Oui, avec `verify`	Non
Idéal pour	Services externes, appels à vérifier	Repositories, données manipulées

QUESTION

À vous : le stock suffit, Alice est débitée de 12 €

Programmez le stock, passez la commande, vérifiez le paiement.



EXERCICE

Feature

- Faire un virement d'un montant X entre deux comptes
- Retirer X au compte source et ajouter X au compte cible
- Le montant du virement doit être positif
- Si le compte source finit avec moins de 0, on envoie un montant de 0,5 vers le compte de la banque

EXERCICE

Play time

EXERCICE

Play time Angular

ACTE 3

Les tests d'intégration

Tester avec le framework et la base

Le principe

- Lancer tout ou une partie de l'application
- Avec une vraie base de données
- Quelques faux, seulement pour les services externes

Pourquoi ?

- Tester la communication entre les classes
- Tester la communication avec le framework
- Vérifier les requêtes sur la vraie base

Alors, que des tests d'intégration ?

- C'est long à exécuter
- C'est plus compliqué à maintenir
- C'est plus compliqué à déboguer

Trois niveaux

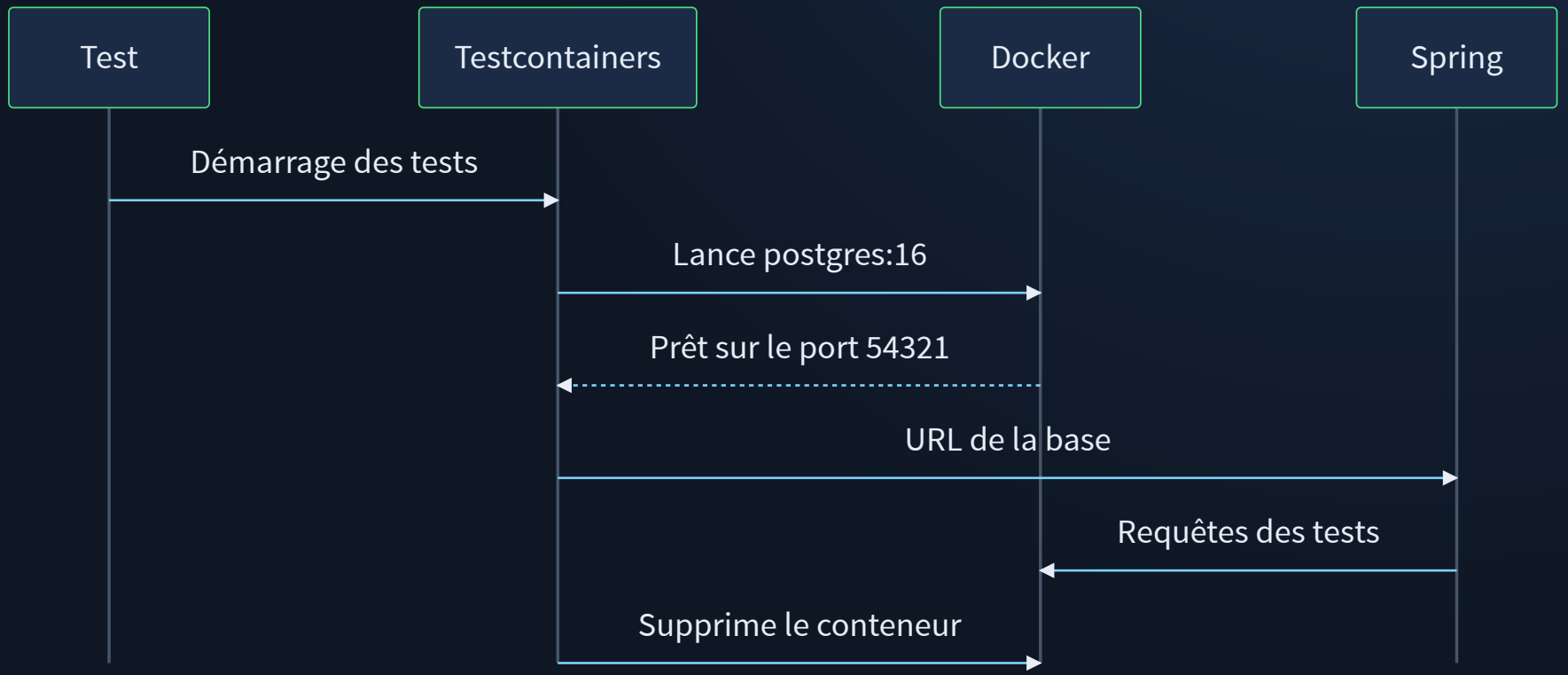
Test de…	Ce qu'on vérifie	Outil
Repository	Les requêtes sur la base	`@DataJpaTest`
Service	Le framework, la base, les transactions	`@SpringBootTest`
Contrôleur	L'URL, le JSON, la sécurité, les codes HTTP	`@WebMvcTest`

H2 : la base qui n'est pas PostgreSQL

- Longtemps, on a utilisé **H2** : une base SQL en mémoire
- ⊕ Rapide, rien à installer
- ⊖ Ce n'est pas PostgreSQL : types, fonctions et SQL diffèrent
- ⊖ Test vert sur H2, erreur en production

Testcontainers

Un vrai PostgreSQL, dans Docker, le temps des tests



Test de repository

```java
@DataJpaTest
@Testcontainers
@AutoConfigureTestDatabase(replace = Replace.NONE)
class ProduitRepositoryTest {

    @Container
    @ServiceConnection
    static PostgreSQLContainer<?> postgres =
            new PostgreSQLContainer<>("postgres:16");

    @Autowired
    private ProduitRepository produitRepository;

    @Test
    void trouveLesProduitsEnRupture() {
        produitRepository.save(new Produit("tomate", 0));
        produitRepository.save(new Produit("comté", 4));

        List<Produit> enRupture = produitRepository.findByStock(0);

        assertEquals(1, enRupture.size());
        assertEquals("tomate", enRupture.get(0).getNom());
```

Et pour un middleware ?

RabbitMQ, Kafka, Redis… : pareil, avec Testcontainers

```
@Container
@ServiceConnection
static RabbitMQContainer rabbit =
        new RabbitMQContainer("rabbitmq:4-management");
```

Test de contrôleur

1. `@WebMvcTest` ne démarre que la couche web
2. Le service est un mock : on teste le mapping, pas la règle métier
3. `MockMvc` simule une requête HTTP, sans vrai serveur
4. On vérifie le code HTTP et le JSON

Test de contrôleur

```java
@WebMvcTest(CommandeController.class)
class CommandeControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockitoBean
    private CommandeService commandeService;

    @Test
    void renvoieLaCommandeEnJson() throws Exception {
        when(commandeService.trouver(42))
                .thenReturn(new CommandeDto(42, "alice", 1200));

        mockMvc.perform(get("/api/commandes/42"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.client").value("alice"))
                .andExpect(jsonPath("$.montant").value(1200));
    }
}
```

QUESTION

Un test qui démarre l'application avec un PostgreSQL Testcontainers et appelle `GET /api/produits` en JSON, c'est un test…

A. unitaire

B. d'intégration

C. end-to-end

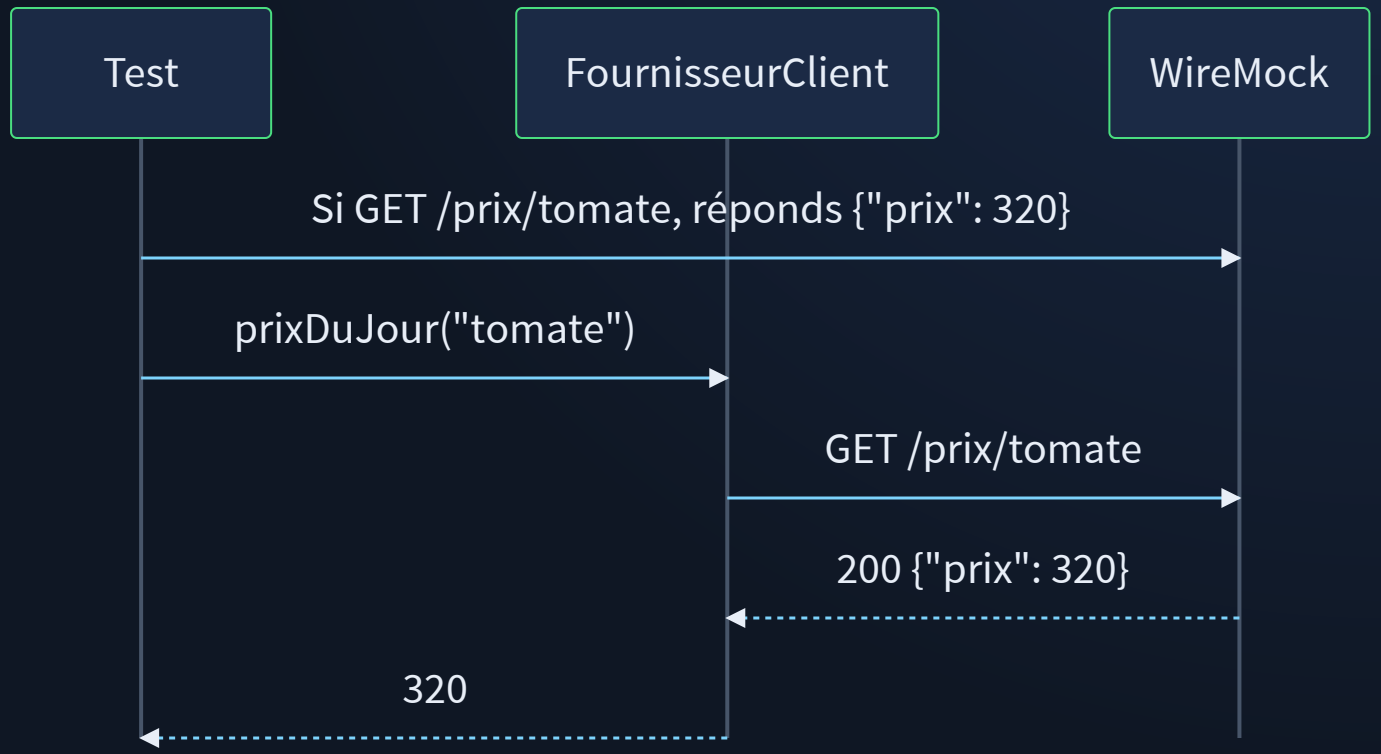
Un client HTTP vers un fournisseur

FraisDirect lit le prix du jour sur l'API de la ferme

Jamais de vraie requête vers un serveur externe dans un test

- Le serveur peut être lent ou en panne : le test échoue sans raison
- L'API peut être payante, ou créer de vraies commandes

WireMock : un faux serveur



WireMock : le test

```java
@WireMockTest
class FournisseurClientTest {

    @Test
    void litLePrixDuJour(WireMockRuntimeInfo wiremock) {
        stubFor(get("/prix/tomate")
                .willReturn(okJson("{\"prix\": 320}")));
        var url = wiremock.getHttpBaseUrl();
        var client = new FournisseurClient(url);

        long prix = client.prixDuJour("tomate");

        assertEquals(320, prix);
    }
}
```

ACTE 4

Les tests end-to-end

Tester comme un utilisateur

Le principe

1. Lancer l'application complète : front, back, middlewares
2. Simuler des actions dans un navigateur
3. Vérifier ce que l'utilisateur voit

Objectif : vérifier que **tout communique** correctement

Un scénario

1. Ouvrir FraisDirect
2. Ajouter deux comtés à 27 € au panier
3. Ouvrir le panier
4. Lire « Livraison offerte »

Exemple avec Playwright

```
1  test('livraison offerte dès 50 €', async ({ page }) => {
2    await page.goto('http://localhost:4200');
3    const comte = page.getByRole('button',
4        { name: 'Ajouter Comté 18 mois' });
5    await comte.click();
6    await comte.click();
7    await page.getByRole('link', { name: 'Panier' }).click();
8    await expect(page.getByText('Livraison offerte'))
9        .toBeVisible();
10 });
```

Selenium et Cypress fonctionnent sur le même principe

Les difficultés

- Préparer la base
- Lourd à écrire
- Lourd à exécuter
- Quand les exécuter ?
- Difficile à maintenir

Faut-il en faire ?

- ⊕ **Oui !**
- ⊕ Au moins sur les chemins usuels : commander, payer, se connecter
- ⊖ Ils ne remplacent pas les tests manuels

À retenir

- Tester, c'est surtout éviter les **régressions**
- TDD : rouge, vert, refactor, par petits pas
- Unitaire : une classe, des mocks et des fakes, tester ses décisions
- Intégration : une vraie base avec Testcontainers, jamais de vrai serveur externe
- End-to-end : peu, sur les chemins qui comptent